```
ENTITY entity_name IS
  [ GENERIC (
    [SIGNAL]  identifier  {, identifier}: [MODE] signal_type
      [ :=static_expression ]
    {; [SIGNAL]  identifier  {,identifier} : [MODE] signal_type
      [ := static_expression ]} );   ]
  PORT (
   [SIGNAL]  identifier  {, identifier}: [MODE] signal_type
   {; [SIGNAL]  identifier  {,identifier} : [MODE] signal_type} ):
  END [ENTITY]  [entity_name];
```

```
ARCHITECTURE architecture_name OF entity_name IS
   Type_declaration
    | signal_declaration ;
    | constant_declaration ;
    | component_declaration ;
    | alias_declaration ;
    | attribute_specification ;
    | subprogram_body ;
  BEGIN
    { process_statement
        | concurrent_statement ;
        | component_instantiation_statement ;
        | generate_statement ;
  END [ARCHITECTURE] [architecture_name];
```

```
CONFIGURATION configuration_name OF entity_name IS
   FOR architecture_name
     { FOR instantiation_label :  component_name
         USE ENTITY entity_name ( architecture_name );
       END FOR; }
   END FOR;
END [CONFIGURATION] [configuration_name ];
```

```
PACKAGE package_name IS
  Type_declaration
    | signal_declaration ;
    | constant_declaration ;
    | component_declaration ;
    | alias_declaration ;
    | attribute_specification ;
    | subprogram_body ;
END [PACKAGE] [package_name];
```

```
COMPONENT  component_name IS
   [ GENERIC (
      [SIGNAL]  identifier {,identifier}:[MODE] signal_type
        [ :=static_expression ]
      {; [SIGNAL] identifier {,identifier}:[MODE] signal_type
        [ :=static_expression]} );  ]
PORT (
    [SIGNAL]  identifier {,identifier}:[MODE] signal_type
    {; [SIGNAL] identifier {,identifier}:[MODE] signal_type } );
END COMPONENT [component_name];
```

```
Instantiation_label: Component_name
   [ GENERIC MAP (
        signal_name | expression | variable_name | OPEN
        {, signal_name | expression | variable_name | OPEN }) ]
PORT MAP (
     signal_name | expression | variable_name | OPEN
     {, signal_name | expression | variable_name | OPEN }) ;
```

```
Instantiation_label: Component_name
   [GENERIC MAP ( generic_name =>
       signal_name | expression | variable_name | OPEN
     {,generic_name=>
       signal_name | expression | variable_name | OPEN }) ]

PORT MAP ( port_name =>
    signal_name | expression | variable_name | OPEN
   {, port_name=>
     signal_name | expression | variable_name | OPEN })  ;
```

CONCURRENT STATEMENTS

```
{ identifier <=expression; }

identifier <=expression WHEN condition ELSE expression
            {expression  WHEN condition ELSE expression }
            {expression WHEN OTHERS } ;

WITH select_expression SELECT
    Identifier <=expression WHEN choices
                { expression WHEN choices }
                { expression WHEN OTHERS } ;

generate_label:
    | FOR identifier in discrete_range
    | IF condition
    GENERATE
        { concurrent statement ; }
    END GENERATE [generate_label];

[ Process_label: ] PROCESS (sensitivity_list)
     {type_declaration
      | constant_declaration;
      | variable_declaration;
      | alias_declaration; }
BEGIN
   { | wait_statement;
     | signal_assignment_statement;
     | variable_assignment_statement;
     | if_statement;
     | case-statement;
     | loop_statement; }
END PROCESS [ process_label ];

FUNCTION identifier [ parameter_list]
     RETURN signal_type  IS
     { declarative_item }
BEGIN
   { sequential_statement }
END [ FUNCTION ] [ identifier ];

PROCEDURE identifier [ parameter_list] IS
    { declarative_item }
BEGIN
   { sequential_statement }
END [ PROCEDURE ] [ identifier ];
```

SEQUENTIAL STATEMENTS

IF condition **THEN** sequence of statements
{ **ELSIF** condition **THEN** sequence of statements}
[**ELSE** sequence of statements]
END IF;

CASE expression **IS**
{ **WHEN** identifier | expression | discrete_range | **OTHERS**
=> sequence of statements }
END CASE;

[loop_label] **LOOP**
{ sequence if statements }
END LOOP [loop_label] ;

[loop_label] **FOR** identifier **IN** discrete_range **LOOP**
{ sequence of statements }
END LOOP [loop_label];

[loop_label] **WHILE** condition **LOOP**
{ sequence of statements }
END LOOP [loop_label];

WAIT

WAIT ON signal { , signal } ;

WAIT UNTIL condition;

WAIT FOR time_constant;

DATA OBJECTS

SIGNAL
CONSTANT
VARIABLE

DATA TYPES

BIT values are ‘0’ and ‘1’

BIT_VECTOR values are array of bits: “01010001”
Example: **SIGNAL** X : BIT_VECTOR(7 DOWNTO 0);

BOOLEAN value **TRUE** and **FALSE**

STD_ULOGIC
STD_LOGIC values ‘U’, ‘X’, ‘0’, ‘1’, ‘Z’, ‘W’, ‘L’, ‘H’, ‘-‘

Enumeration type
Example: **TYPE** states **IS** (reset, S1, S2, s3);

INTEGER
REAL

DATA TYPES (continued)

SIGNED
UNSIGNED
PHYSICAL

OPERATORS

Absolute	**ABS**
Exponentiation	**
Multiplying	* / **MOD** **REM**
Unary	+ -
Addition	+ -
Concatenation	&
Shift	**SLL** **SRL** **SLA** **SRA** **ROL** **ROR**
Relational	= != < <= > >=
Logical	**NOT** **AND** **OR** **NAND** **NOR** **XOR** **XNOR**

MODES

IN **OUT** **BUFFER** **INOUT**

LIBRARY

LIBRARY IEEE;
USE IEEE.std_logic_1164.all;
Standard for describing interconnection data types for VHDL and the STD_LOGIC and STD_LOGIC_VECTOR types

USE IEEE std_logic_arith.all;
SIGNED and UNSIGNED types, **arithmetic** and **comparison** Functions for use with SIGNED and UNSIGNED types, and the conversion function CONV_INTEGER,CONV_SIGNED, and CONV_UNSIGNED.

USE IEEE.std_logic_signed.all;
Functions that allow Quartus to use STD_LOGIC_VECTOR as if they are SIGNED types.

USE IEEE.std_logic_unsigned.all;
Functions that allow Quartus to use STD_LOGIC_VECTOR as if they are UNSIGNED types.

Notation:	[ ]	optional
	{ }	may be repeated
	|	or
	Bold Case	reserved word
	regular case	user supplied